C200 Programming Assignment № 10

Dr. M.M. Dalkilic
Computer Science
School of Informatics, Computing, and Engineering

Indiana University, Bloomington, IN, USA

April 13, 2024

Introduction

In this homework, you'll work on translating critical thinking to programming. This homework is a bit less intense than the last one! :) Since this is **almost** the last homework, you'll be reading some on your own and experimenting.

Make sure to commit to Github and upload the HW to Autograder by **11:00PM EST, April, 20th, Saturday**

Problem 1: Random Walk

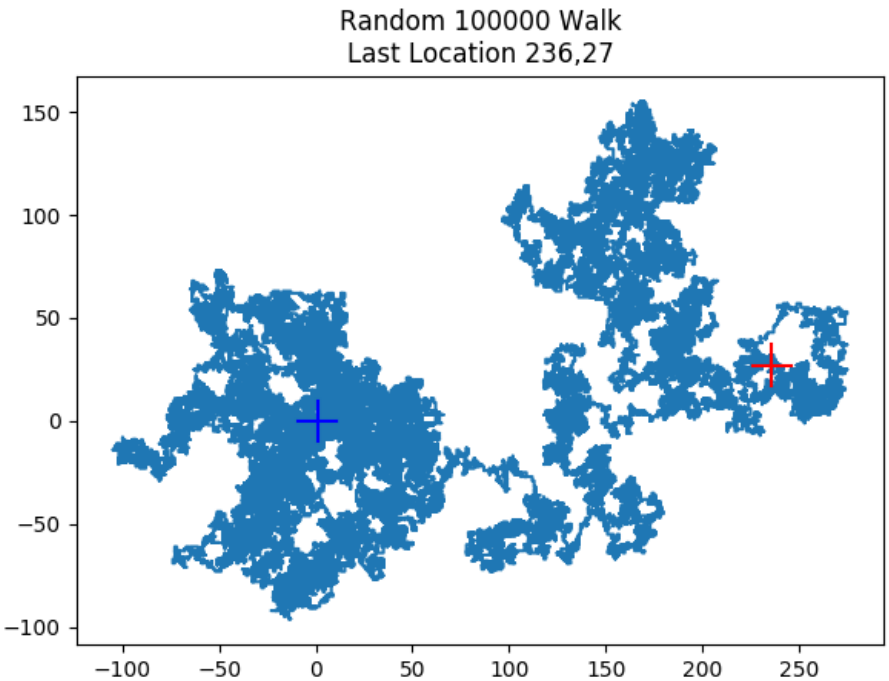

Figure 1: The blue cross is where we started and the red cross is where we ended.

A random walk is a *stochastic process*. A stochastic process is a series of values that are not determined functionally, but probabilistically. The random walk is supposed to describe an inebriated person who, starting from the bar, intends to walk home, but because of intoxication instead randomly takes single steps either forward or backward, left or right. The person has no memory of any steps taken, so theoretically, the person shouldn't move too far from where he or she starts. Random walks are used to model many phenomena, like the size of the web or changes in financial instruments. We will model a 2D random walk with two arrays x and y where x represents moving left or right and y represents forward or backward. The index i will represent the step and x[i],y[i] will represent the location at step i. So, for i=0, we have x[0],y[0] (starting place). Using random we choose from the list [1,2,3,4]. If the value is one then we move right from the previous x position:

```
x[i] = x[i-1] + 1
y[i] = y[i-1]
```

If the value is two, then we move left from the previous x position:

```
x[i] = x[i-1] - 1
y[i] = y[i-1]
```

If the value is three, we move up from the previous y position:

```
x[i] = x[i-1]
y[i] = y[i-1] + 1
```

And when the value is four, we move down from the previous y position:

```
x[i] = x[i-1]
y[i] = y[i-1] - 1
```

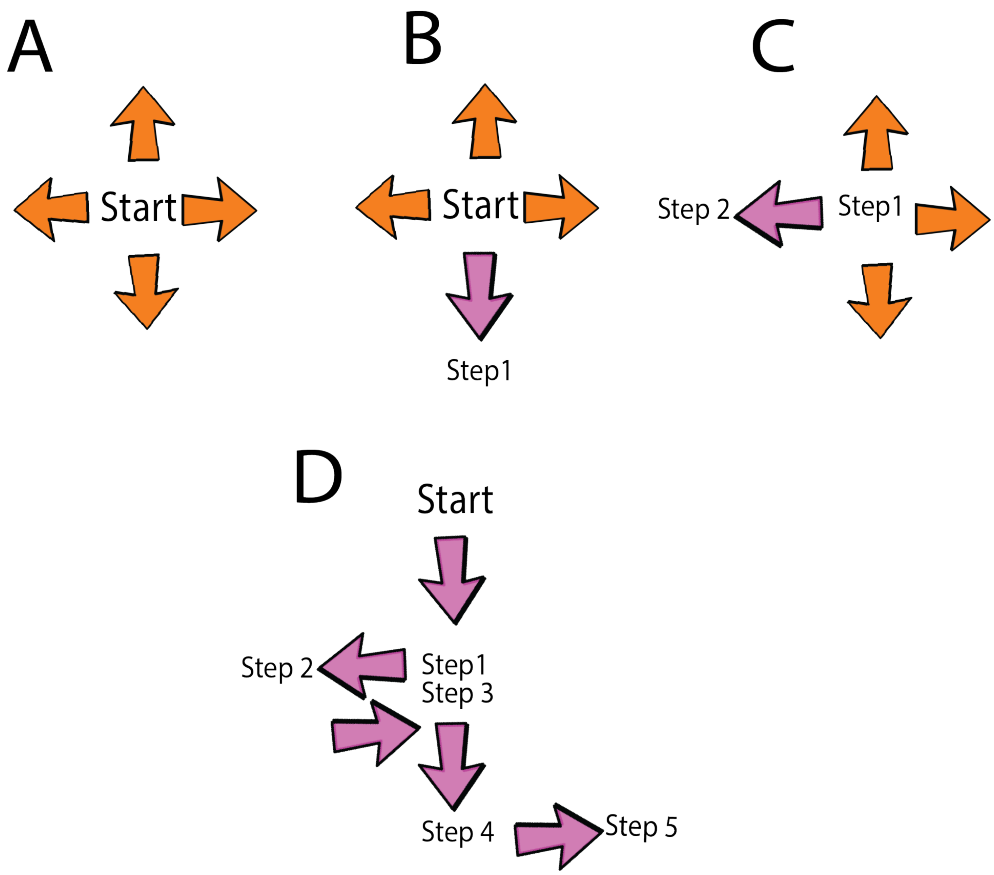

Figure 2: The first few steps of a random walk.

Here is another way to describe this:

$$step(0) = 0 \tag{1}$$

$$step_i(n) = \begin{cases} \text{left} & step(n-1),\ i=1 \\ \text{right} & step(n-1),\ i=2 \\ \text{up} & step(n-1),\ i=3 \\ \text{down} & step(n-1),\ i=4 \end{cases} \tag{2}$$

Session Output

```
Number of Steps: 100000
```

Programming Problem 1: Random Walk

- Complete the `step` function. (Template code has been provided)
- These are random walks, these outputs will be different.
- Experiment with different numbers of steps. Do you see any number of steps that seems to always move significantly away from the start? (Put this answer in a comment at the top of the code)

Problem 2: Imaginary Numbers & Fractals

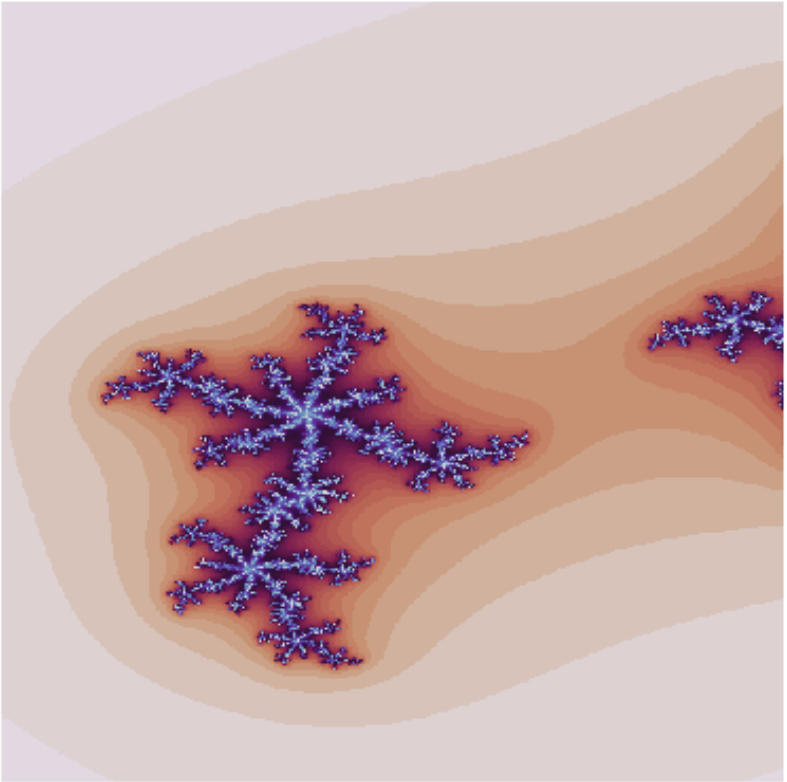

Figure 3: Erie silver snowflakes on an alien planet of sand.

In class we discussed how complex numbers are a pair of real numbers, $a, b \in \mathbb{R}$ and i is the solution to $i^2 = -1$. We called a the *real* part and b the imaginary part. We found that Python has a complex class too, but engineers chose j for i. In this problem, we're extending our complex number class called Complex_Number to include a few more operations:

- modulus overloading __abs__. The modulus of a complex number $x \pm yj$ is $\sqrt{x^2 + y^2}$
- power overload __pow__ that returns the power of a number (see `https://docs.python.org/3/library/operator.html?highlight=__pow__`). For example,

```
>>> x = complex(1,2)
>>> for i in range(6):
...     x**i
...
(1+0j)
(1+2j)
```

```
(-3+4j)
(-11-2j)
(-7-24j)
(41-38j)
>>>
```

We have included * as shown in class. When the class is complete, you'll have a complex number class that you'll use for what I created that looks like snowflakes for this code:

```
def mandelbrot(z,MAX_ITER):
    n = 0
    lz = Complex_Number(0,0)
    c = Complex_Number(-0.1, 0.65)
    while abs(z) <= 10 and n < MAX_ITER:
        z = z**2 + lz + c
        lz = z
        n += 1
    return n/MAX_ITER
```

As you can see from the code, we need modulus, addition, and power. When run, this code:

```
x1 = Complex_Number(1,2)
y1 = complex(1,2)
print(x1,y1)
print(abs(x1),abs(y1))
   for i in range(6):
       print(x1**i,y1**i)
```

produces

```
(1+2j) (1+2j)
2.23606797749979 2.23606797749979
(1+0j) (1+0j)
(1+2j) (1+2j)
(-3+4j) (-3+4j)
(-11-2j) (-11-2j)
(-7-24j) (-7-24j)
(41-38j) (41-38j)
```

and the fractal image. As you can see, I modified the __str__ to mimic Python's print.

Deliverable for Problem 2: Complex Numbers

- Complete the Complex_Number class
- Do *NOT* use any built-in packages for complex numbers (including cmath, complex()). You are making your own `Complex` class.

Problem 3: SQL

In class we were introduced to SQL and the relational model. You will have a great deal of freedom with this problem. Create a table called `Weather` with attributes `City`, `State`, `High`, `Low` and populate it with the data shown in Table 1. I've made equivalent list comprehension on this

Weather

City	State	High	Low
Phoenix	Arizona	105	90
Tucson	Arizona	101	92
Flag Staff	Arizona	105	90
San Diego	California	77	60
Albuquerque	New Mexico	80	72
Nome	Alaska	64	-54

Table 1: Relation Weather and tuples

.

iterable:

```
data = [('Phoenix', 'Arizona', 105, 90),
            ('Tucson', 'Arizona', 101, 92),
            ('Flag Staff', 'Arizona', 105, 90),
            ('San Diego', 'California', 77, 60),
            ('Albuquerque', 'New Mexico', 80, 72),
            ('Nome', 'Alaska', 64 ,-54)]
```

You should read about these SQL functions (**NOTE**: SQL functions, not Python functions): `count()`, `sum()`, `min()`, `max()` as well as "group by" and "in". We've given the answer to query 1 (i.e. MYSQL query). The results of these queries are shown in the output. The following section shows the results obtained by using python and list comprehension-you are required to implement the MYSQL operations in python to implement these queries.

1. Select all the tuples (Query 1) This is an example–observe where the SQL is and replicate this for the remainder of the queries

```
print("Query 1")
for i in my_cursor.execute("SELECT * FROM Weather"):
    print(i)
print("List Comprehension: ", data)
```

2. Select all the tuples where the High temperature is less than 80 (Query 2)

```
print("Query 2")
#for i in my_cursor.execute("Some fantastic SQL"):
#    print(i)
print("List Comprehension: ", [d for d in data if d[2] < 80 ])
```

3. Select All the cities where the low temperature is strictly greater than the Low of Albuquerque – you cannot use the number 72.0 in the query (Query 3)

```
print("Query 3")
#for i in my_cursor.execute("Some fantastic SQL"):
#    print(i)

x =[d[0] for d in data if d[3] > [d[3] for d in data if d[0] == 'Albuquerque'][0]]
print("List Comprehension: ",x)
```

4. Select the city and temperature with the smallest low temperature (Query 4)

```
print("Query 4")
#for i in my_cursor.execute("Some fantastic SQL"):
#    print(i)

print("List Comprehension: ",[(d[0],d[3]) for d in data if d[3] in (sorted(data, key = lambda x:x[3])[0])])
```

5. Select the city temperature with the largest high temperature–since there are two, both cities should be returned. (Query 5)

```
print("Query 5")
#for i in my_cursor.execute("Some fantastic SQL"):
#    print(i)

print("List Comprehension: ",[(d[0],d[2]) for d in data if d[2] in (sorted(data, key = lambda x:x[2],reverse=True)[0])])
```

6. Display the *average* High and Low temperatures–you are not allowed to use Avg() (Query 6)

```
print("Query 6")
#for i in my_cursor.execute("Some fantastic SQL"):
#    print(i)

```

```
print("List Comprehension: ", [(sum([d[2] for d in data])/len(data),sum([d[3] for d in data])/len(data))])
```

7. Give the counts of cities by their Low temperatures (Query 7)

```
print("Query 7")
#for i in my_cursor.execute("Some fantastic SQL"):
#    print(i)

print([(i,list(map((lambda x: x[3]),data)).count(i)) for i in set(map((lambda x: x[3]),data))])
```

Output

```
Query 1
('Phoenix', 'Arizona', 105.0, 90.0)
('Tucson', 'Arizona', 101.0, 92.0)
('Flag Staff', 'Arizona', 105.0, 90.0)
('San Diego', 'California', 77.0, 60.0)
('Albuquerque', 'New Mexico', 80.0, 72.0)
('Nome', 'Alaska', 64.0, -54.0)
List Comprehension:
[('Phoenix', 'Arizona', 105, 90),
 ('Tucson', 'Arizona', 101, 92),
 ('Flag Staff', 'Arizona', 105, 90),
 ('San Diego', 'California', 77, 60),
 ('Albuquerque', 'New Mexico', 80, 72),
 ('Nome', 'Alaska', 64, -54)]

Query 2
('San Diego', 'California', 77.0, 60.0)
('Nome', 'Alaska', 64.0, -54.0)
List Comprehension:
[('San Diego', 'California', 77, 60),
 ('Nome', 'Alaska', 64, -54)]

Query 3
('Phoenix',)
('Tucson',)
('Flag Staff',)
List Comprehension:  ['Phoenix', 'Tucson', 'Flag Staff']
```

Output

```
Query 4
('Nome', -54.0)
List Comprehension:  [('Nome', -54)]

Query 5
('Phoenix', 105.0)
('Flag Staff', 105.0)
List Comprehension:  [('Phoenix', 105), ('Flag Staff', 105)]

Query 6
(88.66666666666667, 58.333333333333336)
List Comprehension:  [(88.66666666666667, 58.333333333333336)]

Query 7
(-54.0, 1)
(60.0, 1)
(72.0, 1)
(90.0, 2)
(92.0, 1)
List Comprehension:  [(72, 1), (-54, 1), (60, 1), (90, 2), (92, 1)]
```

SQL

- Write the MYSQL code using python for queries 1-7.
- Although this is never done, for the purposes of making the problem easier, we will make sure to remove the table and repopulate it everytime so we don't have to comment out portions.

Problem 4: Decision Trees From a Relation (or Table)

One of the most successful techniques in AI is building a function $f : X \to Y$. We are given data in the form

$$data = \{((x_{00}, x_{01}, \ldots, x_{0m}), y_0), ((x_{10}, x_{11}, \ldots, x_{1m}), y_1), \ldots, ((x_{n0}, x_{n1}, \ldots, x_{n1}), y_n) \quad (3)$$

as a relation: where TID is the tuple ID and each row is called a *tuple*. X collectively are known

TID	X_0	X_1	$\cdots$	X_m	Y
0	x_{00}	x_{01}	$\cdots$	x_{0m}	y_0
1	x_{10}	x_{11}	$\cdots$	x_{1m}	y_1
$\vdots$	$\vdots$				
n	x_{n0}	x_{n1}	$\cdots$	x_{n1}	y_n

as the *features* and Y the label. In database parlances X, Y are known as attributes. Given the relation (or data) we build a *decision tree* that's composed from a series of features and values. The function we'll be building in this problem is for is this data:

A Person experienced 90 days past due delinquency or worse as 0/1 (Boolean)

B Total balance on credit cards and personal lines of credit except real estate and no installment debt like car loans divided by the sum of credit limits as percentage (float)

C Age of borrower in years as integer

D Number of times borrower has been 30-59 days past due but no worse in the last 2 years as integer

E Monthly debt payments, alimony,living costs divided by monthy gross income as percentage (float)

F Monthly income as float

G Number of Open loans (installment like car loan or mortgage) and Lines of credit (e.g. credit cards) as integer

H Number of times borrower has been 90 days or more past due integer as

I Number of mortgage and real estate loans including home equity lines of credit as integer

J Number of times borrower has been 60-89 days past due but no worse in the last 2 years as integer

K Number of dependents in family excluding themselves (spouse, children etc.) as integer

Here are three tuples: So, the first tupe indicates the person was delinquent of at least a

A	B	C	D	E	F	G	H	I	J	K
1	0.766126609	45	2	0.802982129	9120	13	0	6	0	2
0	0.957151019	40	0	0.121876201	2600	4	0	0	0	1
0	0.65818014	38	1	0.085113375	3042	2	1	0	0	0

month (Feature A's definition), but the other two were not. We want to build a function f : BCDEFGHIJK $\rightarrow A$ so that when we see a person's history:

$$[0.025892469, 38, 0, 74.5, 1, 13, 0, 0, 0, 2] \tag{4}$$

$$[0.762158561, 44, 4, 0.224465502, 21000, 5, 0, 1, 0, 3] \tag{5}$$

we can predict if there will be a likely delinquency at least a month. A decision tree will build a structure that will try to solve this problem. We will have spoken about this in lecture, but you should visit: `https://scikit-learn.org/stable/modules/tree.html` and run the example with the iris data with which you are familiar. The tree can be visualized as shown in Fig. 4:

The following code:

```
tree_model = build_tree_model("cr.csv")
unknown = pd.DataFrame(data = [[0.025892469,38,0,74.5,1,13,0,0,0,2],
                               [0.762158561,44,4,0.224465502,21000,5,0,1,0,3]],↩
                                       columns = ['B', 'C', 'D', 'E', 'F', ↩
                                   'G', 'H', 'I', 'J ', 'K'])
print(tree_model.predict(unknown))  #1,0
tree.plot_tree(tree_model)
plt.show()
```

produces:

```
[1 0]
```

which means the first person will, according to the data, be deliquent, and the second one will not. Your function will take a string (name of the file), and use pandas to do the following:

- Clean the data by removing rows that have Na (not a number). We covered this in lecture, but as hint, look for dropna.
- Separate the table into two data frames: B-K (which are the features) and A which is the label or target. You build the tree as shown in the url.
- Return the the fitted tree model

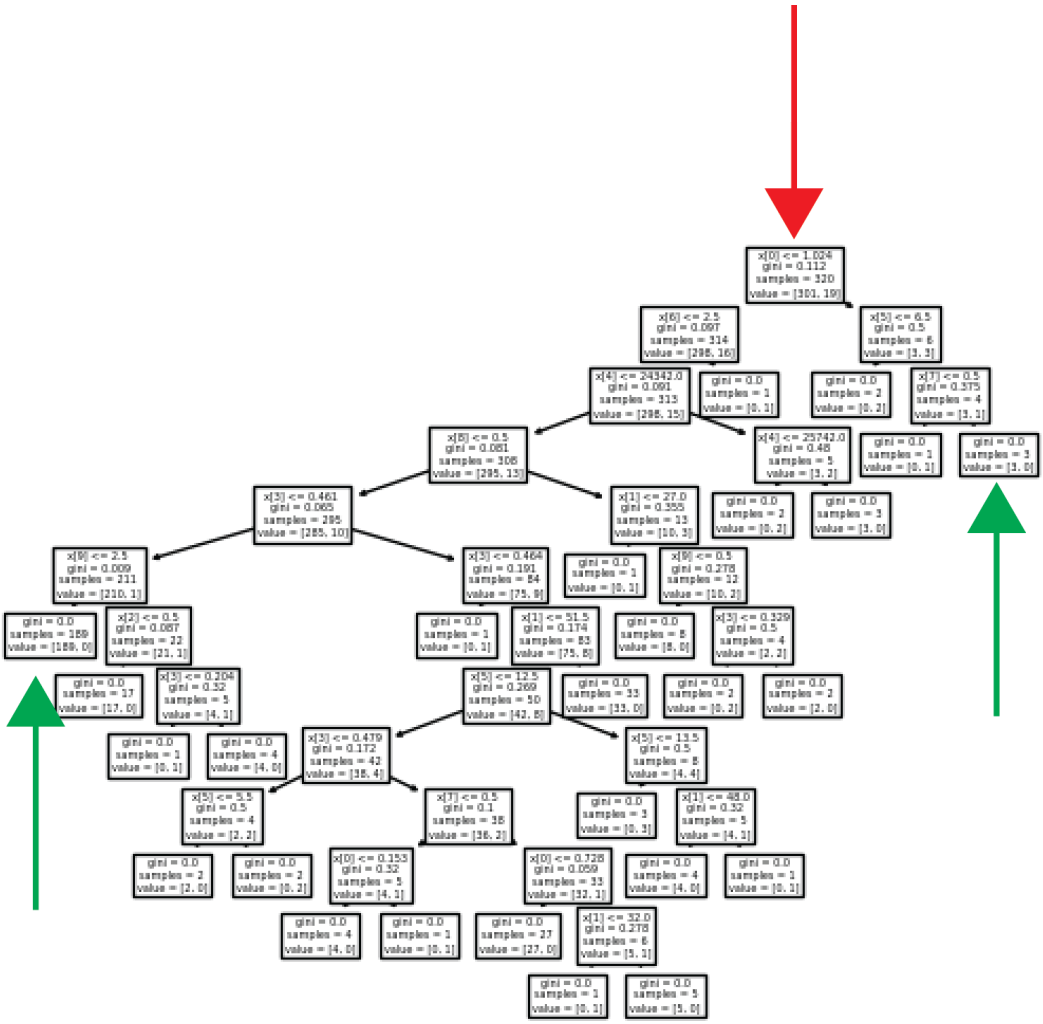

Figure 4: Red points to where data starts (root); green points to a possible end (leaf).

Decision Tree

- Implement the build_tree_model. The tasks are described above, but your task is to return the fitted model.
- Confirm that your prediction is 1,0 for the pairs of unknown, respectively.
- If your model is build correctly, you'll be able to visualize it.
- Remember there was a condition for dropna to be committed.

Programming Pairs

achitred@iu.edu, gravipat@iu.edu
ml82@iu.edu, lawlin@iu.edu

emiellis@iu.edu, rimaeda@iu.edu
njustin@iu.edu, trimedio@iu.edu
gmhowell@iu.edu, corrosar@iu.edu
presharr@iu.edu, camyee@iu.edu
sjbarry@iu.edu, masoroki@iu.edu
cdinman@iu.edu, peterabl@iu.edu
craftdc@iu.edu, bpovinel@iu.edu
abelani@iu.edu, smremmer@iu.edu
samjimen@iu.edu, alexpan@iu.edu
rcerauli@iu.edu, bsai@iu.edu
jailjohn@iu.edu, amatariy@iu.edu
marbelli@iu.edu, ap59@iu.edu
hedavid@iu.edu, mxrobins@iu.edu
harmonnc@iu.edu, lomwean@iu.edu
nsfried@iu.edu, javitto@iu.edu
blevidut@iu.edu, aquintas@iu.edu
cbremner@iu.edu, wangjinz@iu.edu
langgrav@iu.edu, jlopezmo@iu.edu
kespejo@iu.edu, etmillho@iu.edu
changali@iu.edu, archshar@iu.edu
careyjo@iu.edu, cmm23@iu.edu
ykassahu@iu.edu, weinstab@iu.edu
logellis@iu.edu, surehma@iu.edu
chenkay@iu.edu, ksreejit@iu.edu
mbadwe@iu.edu, arangi@iu.edu
balian@iu.edu, jamread@iu.edu
blberger@iu.edu, reddysk@iu.edu
shaagore@iu.edu, olipfing@iu.edu
chamblia@iu.edu, joebrahm@iu.edu
nabatja@iu.edu, rsreedh@iu.edu
anchinna@iu.edu, xsoto@iu.edu
sambereb@iu.edu, abbalokh@iu.edu
sebcont@iu.edu, gstogdil@iu.edu
anudivek@iu.edu, sparab@iu.edu
pafrost@iu.edu, johnsanc@iu.edu
akumares@iu.edu, wardjohn@iu.edu
ejbusch@iu.edu, camlueck@iu.edu
cjduffie@iu.edu, bfmark@iu.edu
agunde@iu.edu, menonvi@iu.edu
arkali@iu.edu, ggriorda@iu.edu
camehugh@iu.edu, jianlou@iu.edu

aydclink@iu.edu, smunikun@iu.edu
lineri@iu.edu, tmukund@iu.edu
lasdodd@iu.edu, kammich@iu.edu
jdemian@iu.edu, nmetlapa@iu.edu
ecanant@iu.edu, vupatel@iu.edu
jakedeck@iu.edu, zstopher@iu.edu
wchorich@iu.edu, arstover@iu.edu
allentp@iu.edu, tryerrag@iu.edu
beckab@iu.edu, ymarrakc@iu.edu
belleli@iu.edu, nsenthi@iu.edu
jkchaura@iu.edu, saravula@iu.edu
adeglur@iu.edu, traepena@iu.edu
andhodge@iu.edu, ceprenti@iu.edu
cfahy@iu.edu, ntorpoco@iu.edu
tkonkim@iu.edu, snlnu@iu.edu
conkeen@iu.edu, jomalek@iu.edu
cedonica@iu.edu, avaweish@iu.edu
ocrowder@iu.edu, haoztian@iu.edu
sgudapur@iu.edu, jmatczak@iu.edu
zaycarey@iu.edu, katepate@iu.edu
jaxkatz@iu.edu, breramer@iu.edu
burkhel@iu.edu, shmulla@iu.edu
nkhai@iu.edu, ishetty@iu.edu
shbivens@iu.edu, peterant@iu.edu
nkushnir@iu.edu, amtsmith@iu.edu
pkurzawa@iu.edu, colbstap@iu.edu
allennam@iu.edu, jusshah@iu.edu
mgrass@iu.edu, khwillar@iu.edu
ankotel@iu.edu, mitcchar@iu.edu
sdharmmi@iu.edu, rahkpate@iu.edu
jvgottum@iu.edu, tiffyou@iu.edu
lrborows@iu.edu, andmuchi@iu.edu
renahan@iu.edu, nicmill@iu.edu
nbkotur@iu.edu, shubtiwa@iu.edu
ryandaun@iu.edu, wagnebr@iu.edu
arkhal@iu.edu, leoett@iu.edu
boding@iu.edu, joprayit@iu.edu
adevill@iu.edu, mackeyr@iu.edu
forlin@iu.edu, mvsimi@iu.edu
rachguo@iu.edu, adpulipa@iu.edu
tkalale@iu.edu, ashahi@iu.edu

aadkhan@iu.edu, brshelby@iu.edu
madimond@iu.edu, benserna@iu.edu
cc96@iu.edu, pratmodi@iu.edu
cgreywal@iu.edu, auspiege@iu.edu
askaul@iu.edu, animerch@iu.edu
jhbruce@iu.edu, msolomie@iu.edu
reychaud@iu.edu, twirges@iu.edu
amgandhi@iu.edu, njvinson@iu.edu
daveanu@iu.edu, crschuit@iu.edu
zcarmich@iu.edu, nvoneper@iu.edu
auconner@iu.edu, ctsvetan@iu.edu
harrisdg@iu.edu, cmoverfe@iu.edu
ecabebe@iu.edu, jocpoiri@iu.edu
okorkmaz@iu.edu, syeshala@iu.edu
kimyouh@iu.edu, frvisco@iu.edu
jamlian@iu.edu, nsmurthy@iu.edu
reafrank@iu.edu, mgstack@iu.edu
berrynm@iu.edu, ptapiaro@iu.edu
chodgman@iu.edu, teswank@iu.edu
bcabral@iu.edu, nrimawi@iu.edu
zlangley@iu.edu, bwmoore@iu.edu
markdeng@iu.edu, iopfings@iu.edu